Technologie w programowaniu grafiki komputerowej

Języki programowania w grafice komputerowej – C++

- **Charakterystyka**: Język niskopoziomowy, oferuje kontrolę nad pamięcią i procesami, co jest istotne w grafice 3D.
- **Zastosowania**:
  - Tworzenie silników graficznych (np. Unreal Engine, CryEngine).
  - Rendering wysokiej jakości obrazów.
  - Obsługa bibliotek graficznych, takich jak OpenGL, Vulkan.

Języki programowania w grafice komputerowej – Python

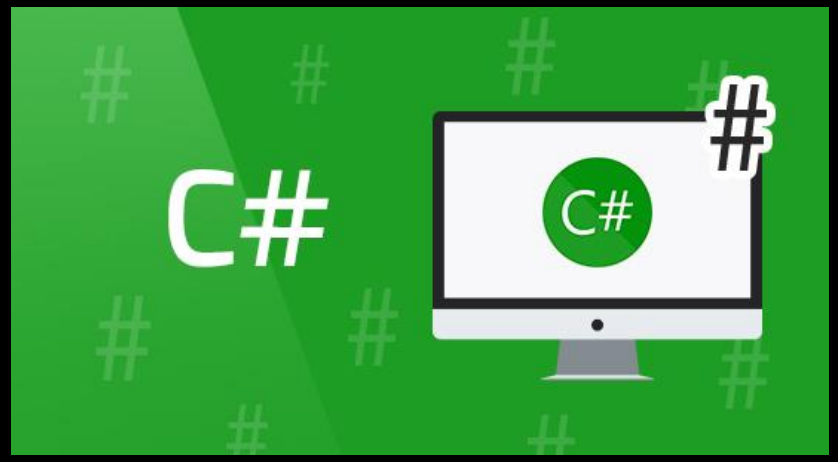
- **Charakterystyka**: Język wysokopoziomowy, łatwy w nauce, z bogatym zestawem bibliotek do grafiki.
- **Zastosowania**:
  - Prototypowanie wizualizacji 2D/3D.
  - Automatyzacja procesów graficznych (np. w Blenderze).
  - Wizualizacja naukowa i danych.
- **Biblioteki**:
  - **Pygame** – tworzenie gier 2D.
  - **Matplotlib** – wizualizacja danych.
  - **Blender API** – automatyzacja renderowania.

Języki programowania w grafice komputerowej – JavaScript

- **Charakterystyka**: Język szeroko używany w grafice przeglądarkowej.
- **Zastosowania**:
  - Tworzenie interaktywnych wizualizacji i gier przeglądarkowych.
  - Renderowanie 3D w przeglądarce.
- **Biblioteki**:
  - **Three.js** – generowanie scen 3D.
  - **Babylon.js** – silnik grafiki 3D.

Języki programowania w grafice komputerowej – C#

- **Charakterystyka**: Język zorientowany obiektowo, popularny w tworzeniu gier.
- **Zastosowania**:
  - Tworzenie gier i aplikacji VR/AR.
  - Programowanie w silniku Unity.



Języki programowania w grafice komputerowej – GLSL i HLSL (języki shaderów)

- **Charakterystyka**: Języki dedykowane programowaniu GPU.
- **Zastosowania**:
  - Tworzenie efektów wizualnych (np. cieni, odbić, tekstur).
  - Pisanie programów do obliczeń równoległych na GPU.

Wykorzystywane platformy w grafice komputerowej - OpenGL (Open Graphics Library)

- **Charakterystyka**:

Popularne, wieloplatformowe API do renderowania grafiki 2D i 3D.

- **Zastosowania**:
  - Tworzenie gier i aplikacji graficznych.
  - Wizualizacje naukowe i inżynierskie.
- **Zalety**:
  - Wieloplatformowość (Windows, macOS, Linux).
  - Otwarty standard i duża społeczność.
- **Przykład zastosowania**:

Renderowanie modeli 3D w programach CAD.

Wykorzystywane platformy w grafice komputerowej -DirectX

- **Charakterystyka**: Zestaw API opracowany przez Microsoft, zorientowany na grafikę 3D i multimedia.
- **Zastosowania**:
  - Gry komputerowe na platformy Windows i Xbox.
  - Aplikacje wymagające wysokiej wydajności grafiki.
- **Zalety**:
  - Wysoka wydajność i optymalizacja pod system Windows.
  - Wsparcie dla zaawansowanych efektów graficznych.
- **Przykład zastosowania**: Gry AAA, takie jak seria *Forza Horizon*.

Wykorzystywane platformy w grafice komputerowej - Vulkan

- **Charakterystyka**: Nowoczesne API graficzne oferujące niskopoziomową kontrolę nad sprzętem.
- **Zastosowania**:
  - Gry i aplikacje wymagające dużej wydajności.
  - Tworzenie efektów graficznych w czasie rzeczywistym.
- **Zalety**:
  - Wysoka wydajność dzięki optymalizacji dla GPU.
  - Wieloplatformowość (Windows, Linux, Android).
- **Przykład zastosowania**: Gry na Androida, takie jak *DOOM Eternal*.

Wykorzystywane platformy w grafice komputerowej - Unity

- **Charakterystyka**: Popularny silnik do tworzenia gier, wspierający zarówno 2D, jak i 3D.
- **Zastosowania**:
  - Gry na komputery, konsole, urządzenia mobilne i VR.
  - Aplikacje wizualizacyjne i symulacje.
- **Zalety**:
  - Łatwość obsługi i intuicyjny interfejs.
  - Wsparcie dla wielu platform.
- **Przykład zastosowania**: Tworzenie gier mobilnych, takich jak *Monument Valley*.

Wykorzystywane platformy w grafice komputerowej – Unreal Engine

- **Charakterystyka**: Zaawansowany silnik graficzny, używany głównie w grach AAA i wizualizacjach.
- **Zastosowania**:
  - Tworzenie gier 3D i aplikacji VR/AR.
  - Renderowanie realistycznych wizualizacji architektonicznych.
- **Zalety**:
  - Niesamowita jakość grafiki dzięki technologii Lumen i Nanite.
  - Wsparcie dla dużych projektów.
- **Przykład zastosowania**: Gry takie jak *Fortnite* i wizualizacje architektoniczne.

Technologie w grafice komputerowej.

- Grafika komputerowa opiera się na wykorzystaniu zaawansowanych technologii, które umożliwiają tworzenie i wyświetlanie obrazów 2D i 3D. Poniżej zostaną przedstawione niektóre z nich.

Silniki graficzne

- Silniki graficzne to kompleksowe narzędzia umożliwiające tworzenie gier i aplikacji wizualnych. Zawierają wbudowane systemy renderowania, fizyki, animacji i wiele innych funkcji.
- Przykłady: Unity, Unreal Engine

API graficzne

- API (Application Programming Interface) to interfejsy umożliwiające komunikację między oprogramowaniem a sprzętem graficznym.
- **DirectX**: Stworzony przez Microsoft, wykorzystywany w grach na Windows i Xbox.
- **OpenGL**: Otwarty standard graficzny, popularny w aplikacjach wieloplatformowych.
- **Vulkan**: Nowoczesne API zapewniające wysoką wydajność i niskopoziomowy dostęp do GPU.
- **Metal**: API Apple’a, zoptymalizowane pod macOS i iOS

Shadery

- Shadery to małe programy uruchamiane na GPU, które określają sposób wyświetlania pikseli, wierzchołków i innych elementów graficznych.
- **Vertex Shader**: Przetwarza wierzchołki obiektów 3D, np. ich położenie.
- **Fragment Shader (Pixel Shader)**: Odpowiada za kolor, cienie i inne efekty wizualne dla każdego piksela.
- **Geometry Shader**: Pozwala generować nowe wierzchołki na podstawie istniejących.

Ray Tracing (Śledzenie promieni)

- Technologia renderowania, która symuluje sposób, w jaki światło odbija się od obiektów.
- **Zastosowania**:
  - Realistyczne oświetlenie i cienie w grach.
  - Wizualizacje architektoniczne.
- **Przykłady**: Karty graficzne NVIDIA RTX wspierają ray tracing w czasie rzeczywistym.

Technologie VR i AR

- **Virtual Reality (VR)**: Tworzenie w pełni immersyjnych środowisk 3D, używanych w grach, symulacjach czy medycynie.
- **Augmented Reality (AR)**: Nakładanie elementów graficznych na rzeczywisty świat, np. w aplikacjach mobilnych i produktach takich jak Microsoft HoloLens.

Shader'y i ich dostępność w grafice komputerowej.

- Shadery to specjalne programy uruchamiane na procesorze graficznym (GPU), które definiują sposób renderowania obiektów graficznych w grach, aplikacjach 3D i wizualizacjach. Umożliwiają tworzenie realistycznych efektów graficznych, takich jak oświetlenie, cienie, refleksy czy materiały.

Rodzaje shaderów - Vertex Shader

- **Funkcja**: Przetwarza wierzchołki obiektów 3D, określając ich położenie w przestrzeni oraz modyfikując ich właściwości, np. kolor czy współrzędne tekstur.
- **Zastosowania**: Skalowanie, obracanie, deformacje modeli.
- **Przykład**: Ruch falującego materiału.

Rodzaje shaderów - Fragment Shader (Pixel Shader)

1. **Funkcja**: Przetwarza każdy piksel obrazu, definiując jego kolor, przezroczystość czy inne efekty wizualne.
2. **Zastosowania**: Tworzenie realistycznych cieni, refleksów, efektów mgły.
3. **Przykład**: Symulacja powierzchni wody z odbiciami.

Dostępność shaderów

- Shadery są dostępne w różnych środowiskach i narzędziach, co pozwala twórcom na ich łatwe wykorzystanie. Shadery są kluczowym elementem współczesnej grafiki komputerowej, umożliwiając tworzenie zaawansowanych efektów wizualnych. Ich dostępność i wsparcie w wielu platformach i narzędziach sprawiają, że są szeroko wykorzystywane w branży gier, wizualizacji czy aplikacjach VR i AR.

Pojęcie potoku graficznego.

Potok graficzny (ang. *graphics pipeline*) to sekwencja procesów przetwarzania danych graficznych w celu wygenerowania obrazu na ekranie. Stanowi podstawowy mechanizm działania współczesnych systemów renderowania grafiki komputerowej, szczególnie w grach i aplikacjach 3D. Potok graficzny działa na zasadzie przepływu danych – od geometrii obiektów po piksele na ekranie.

Główne etapy potoku graficznego:

- **Etap aplikacji -** W tym etapie działa program aplikacji, który przygotowuje dane do renderowania.
- **Etap geometrii -** Przetwarzanie obiektów w przestrzeni 3D i ich projekcja na ekran 2D.
- **Rasteryzacja -** Proces przekształcenia obiektów 3D na piksele, które będą wyświetlane na ekranie.
- **Etap pikseli (fragmentów) -** Przetwarzanie danych każdego piksela w celu nadania mu właściwego koloru i innych cech wizualnych.
- **Etap wyjścia (renderowania) -** Ostateczne przekształcenie przetworzonych pikseli na obraz wyświetlany na ekranie.

DZIĘKUJE ZA UWAGĘ

Źródła

- Img1, img2, img3, img4, img5, img6, img7, img8, img9, img10
- Dane potrzebne do prezentacji zostały zebrane przy pomocy AI.
- Wikipedia - shadery
- Wikipedia – potok graficzny